

SISTEMAS DE TIEMPO REAL

- INTRODUCCIÓN A LOS SISTEMAS DE TIEMPO REAL Y A LOS SISTEMAS EMBEDDED (INTEGRADOS).
- CARACTERÍSTICAS DEL SISTEMA.
- CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL.
- IMPLEMENTACIÓN.
- PLANIFICACIÓN DEL PROCESADOR.
- EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL).

INTRODUCCIÓN

- LOS SISTEMAS EMBEBIDOS **VARIAN** CONSIDERABLEMENTE.
- ALGUNOS SON **COMPUTADORAS DE PROPÓSITO GENERAL** QUE EJECUTAN SISTEMAS OPERATIVOS ESTANDAR, COMO UNIX, CON **APLICACIONES DE PROPÓSITO ESPECIAL** PARA IMPLEMENTAR LA FUNCIONALIDAD.
- OTROS SON SISTEMAS HARDWARE CON **SISTEMAS OPERATIVOS EMBEBIDOS DE PROPÓSITO ESPECIAL** QUE SÓLO PROPORCIONAN LA FUNCIONALIDAD DESEADA.
- OTROS SON **DISPOSITIVOS HARDWARE CON CIRCUITOS INTEGRADOS ESPECÍFICOS** DE LA APLICACIÓN:
 - ◆ **ASIC**: APPLICATION SPECIFIC INTEGRATED CIRCUIT.
 - ◆ REALIZAN SUS TAREAS SIN SISTEMA OPERATIVO.

INTRODUCCIÓN A LOS SISTEMAS DE TIEMPO REAL Y A LOS SISTEMAS EMBEDDED (INTEGRADOS)

INTRODUCCIÓN

- LOS SISTEMAS EMBEBIDOS CASI SIEMPRE EJECUTAN **SISTEMAS OPERATIVOS EN TIEMPO REAL**.
- UN **SISTEMA EN TIEMPO REAL** SE USA CUANDO SE HAN ESTABLECIDO **RÍGIDOS REQUISITOS DE TIEMPO** EN LA OPERACIÓN DE UN PROCESADOR O DEL FLUJO DE DATOS.
- SE USAN COMO **DISPOSITIVO DE CONTROL EN UNA APLICACIÓN DEDICADA**:
 - ◆ UNA SERIE DE **SENSORES** PROPORCIONAN LOS **DATOS** A LA COMPUTADORA.
 - ◆ LA COMPUTADORA DEBE **ANALIZAR** LOS DATOS Y, POSIBLEMENTE, **AJUSTAR LOS CONTROLES** CON EL FIN DE MODIFICAR LOS DATOS DE ENTRADA DE LOS SENSORES.

INTRODUCCIÓN

- **SISTEMAS EMBEBIDOS EN TIEMPO REAL**.
- LAS COMPUTADORAS **EMBEBIDAS** SON LAS COMPUTADORAS **PREDOMINANTES** ACTUALMENTE.
- UN SISTEMA **INTEGRADO** ES UN **DISPOSITIVO INFORMÁTICO** QUE **FORMA PARTE DE UN SISTEMA DE MAYOR TAMAÑO**, EN EL QUE LA **PRESENCIA** DE ESE DISPOSITIVO INFORMÁTICO A MENUDO **NO RESULTA OBVIA** PARA EL USUARIO.
- SE **ENCUENTRAN** EN LOS MOTORES DE AUTOMÓVILES, AVIONES, ROBOTS INDUSTRIALES, MAGNETOSCOPIOS, HORNOS DE MICROONDAS, LAVAVAJILLAS, ETC.
- SON SISTEMAS DE **PROPÓSITOS MUY ESPECÍFICOS**.
- LOS SISTEMAS OPERATIVOS PROPORCIONAN **FUNCIONALIDADES LIMITADAS**.
- USUALMENTE DISPONEN DE UNA **INTERFAZ DE USUARIO MUY LIMITADA** O NO DISPONEN DE ELLA EN ABSOLUTO.

INTRODUCCIÓN

- **EJEMPLOS** DE SISTEMAS DE TIEMPO REAL SON LOS QUE:
 - ◆ CONTROLAN **EXPERIMENTOS CIENTÍFICOS**.
 - ◆ LOS DE **IMÁGENES MÉDICAS**.
 - ◆ LOS DE **CONTROL INDUSTRIAL**.
 - ◆ CIERTOS SISTEMAS DE **VISUALIZACIÓN**.
 - ◆ ALGUNOS SISTEMAS DE **INYECCIÓN** DE GASOLINA PARA MOTORES DE AUTOMÓVILES.
 - ◆ ALGUNAS **CONTROLADORAS** DE ELECTRODOMÉSTICOS.
 - ◆ ALGUNOS SISTEMAS DE **ARMAMENTO**.

INTRODUCCIÓN

- ALGUNOS SISTEMAS DE TIEMPO REAL SON SISTEMAS DE SEGURIDAD CRÍTICA:
 - ◆ LA OPERACIÓN INCORRECTA (USUALMENTE DEBIDO A QUE NO SE CUMPLE CON UN REQUISITO DE TEMPORIZACIÓN) PROVOCA ALGÚN TIPO DE "CATÁSTROFE".
 - ◆ EJ.: SISTEMAS ARMAMENTÍSTICOS, LOS DE ANTIBLOQUEO DE LOS FRENOS, LOS DE GESTIÓN DE VUELO, LOS MARCAPASOS.
- LA MAYORÍA DE LOS SISTEMAS INTEGRADOS NO SON SISTEMAS DE SEGURIDAD CRÍTICA:
 - ◆ SI NO SE CUMPLEN LOS REQUISITOS DE TEMPORIZACIÓN, NO HAY CONSECUENCIAS CATASTRÓFICAS, A LO SUMO UN USUARIO INSATISFECHO.
 - ◆ EJ.: LAS MÁQUINAS FAX, LOS HORNOS DE MICROONDAS, LOS RELOJES DE PULSERA, LOS DISPOSITIVOS DE RED COMO CONMUTADORES Y ENCAMINADORES.

INTRODUCCIÓN

¿Qué es un STR?



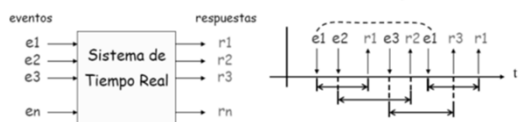
- ✓ Resultados de computación correctos
- ✓ Cumplir restricciones temporales
 - Dar la respuesta a su debido tiempo -> Plazos
 - Realizar una acción cada cierto tiempo -> Periodos

INTRODUCCIÓN

- LA INFORMÁTICA DE TIEMPO REAL PUEDE CLASIFICARSE ASÍ:
 - ◆ SISTEMAS DE TIEMPO REAL ESTRICTO:
 - TIENE LOS REQUISITOS MÁS FUERTES, QUE GARANTIZAN QUE LAS TAREAS DE TIEMPO REAL CRÍTICAS SE COMPLETEN DENTRO DEL PERÍODO ESPECIFICADO.
 - EJ.: LOS SISTEMAS DE SEGURIDAD CRÍTICA.
 - ◆ SISTEMAS DE TIEMPO REAL NO ESTRICTO:
 - NO TIENEN REQUISITOS TAN FUERTES, SE LIMITAN A GARANTIZAR QUE LAS TAREAS DE TIEMPO REAL CRÍTICAS TENGAN PRIORIDAD SOBRE OTRAS TAREAS Y QUE RETENGAN DICHA PRIORIDAD HASTA COMPLETARSE.
 - EJ.: SISTEMAS OPERATIVOS COMERCIALES, ASÍ COMO LINUX, PROPORCIONAN SORTEO DE TIEMPO REAL NO ESTRICTO.

INTRODUCCIÓN

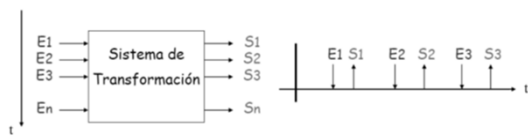
¿Qué es un STR?



- ✓ Resultados de computación correctos
- ✓ Cumplir restricciones temporales
 - Dar la respuesta a su debido tiempo -> Plazos
 - Realizar una acción cada cierto tiempo -> Periodos

INTRODUCCIÓN

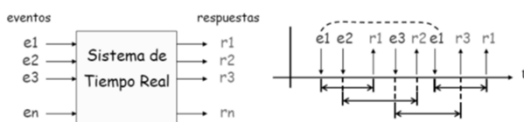
¿Qué es un STR?



- ✓ Resultados de computación correctos

INTRODUCCIÓN

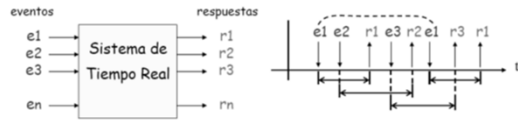
¿Qué es un STR?



- Se abordarán:
- Diseño
 - Implementación
 - Planificación

INTRODUCCIÓN

¿Qué es un STR?



Se abordarán:

Diseño
Implementación
Planificación



SISTEMAS DE TIEMPO REAL

13

INTRODUCCIÓN

¿Qué es un STR?



SISTEMAS DE TIEMPO REAL

16

INTRODUCCIÓN

¿Qué es un STR?



SISTEMAS DE TIEMPO REAL

14

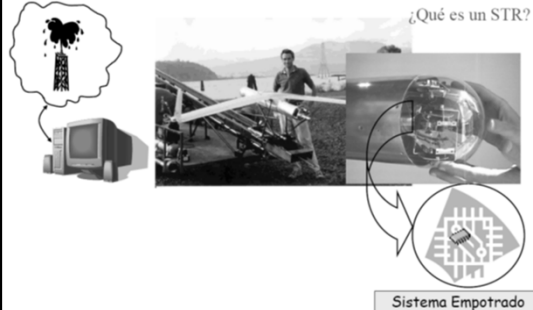
CARACTERÍSTICAS DEL SISTEMA

SISTEMAS DE TIEMPO REAL

17

INTRODUCCIÓN

¿Qué es un STR?



SISTEMAS DE TIEMPO REAL

15

CARACTERÍSTICAS DEL SISTEMA

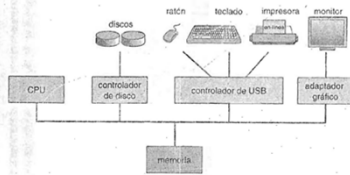
- LAS **CARACTERÍSTICAS TÍPICAS** DE MUCHOS SISTEMAS OPERATIVOS DE TIEMPO REAL SON LAS SIGUIENTES:
 - ◆ TIENEN UN **ÚNICO PROPÓSITO**.
 - ◆ SON DE **PEQUEÑO TAMAÑO**.
 - ◆ SON DE **BAJO COSTE** Y SE PRODUCEN EN MASA.
 - ◆ TIENEN **REQUISITOS DE TEMPORIZACIÓN** ESPECÍFICOS.
- EXISTEN MUCHOS SISTEMAS DE TIEMPO REAL EN ENTORNOS DONDE EL **ESPACIO FÍSICO** ESTÁ **RESTRINGIDO**.
 - ◆ EJ.: UN RELOJ DE PULSERA, UN HORNO DE MICROONDAS.
- ESTO **LIMITA** LA POTENCIA DE PROCESAMIENTO DE LA CPU Y LA CANTIDAD DE MEMORIA.
- MUCHOS SE EJECUTAN SOBRE **PROCESADORES DE 8 O 16 BITS** EN VEZ DE 32 O 64 BITS.
- PUEDEN TENER **MENOS DE UN MEGABYTE** DE MEMORIA EN VEZ DE VARIOS GIGABYTES.
- SE DENOMINA **HUELLA DE UN SISTEMA** A LA CANTIDAD DE **MEMORIA** NECESARIA PARA EJECUTAR **SISTEMA OPERATIVO Y APLICACIONES**: EN ESTOS SISTEMAS ES **PEQUEÑA**.

SISTEMAS DE TIEMPO REAL

18

CARACTERÍSTICAS DEL SISTEMA

- A MENUDO ESTOS SISTEMAS ESTÁN EN ELECTRODOMÉSTICOS Y DISPOSITIVOS DE CONSUMO:
 - ◆ DEBEN FABRICARSE EN MASA Y A BAJO COSTO.
- UNA FORMA DE REDUCIR EL COSTO DE UN CONTROLADOR INTEGRADO CONSISTE EN EMPLEAR UNA TÉCNICA ALTERNATIVA PARA ORGANIZAR LOS COMPONENTES DEL SISTEMA INFORMÁTICO.
- NO SE UTILIZA LA ORGANIZACIÓN ORIENTADA A BUS:

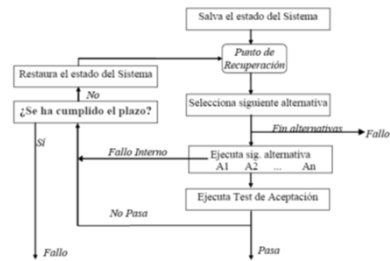


SISTEMAS DE TIEMPO REAL

19

CARACTERÍSTICAS DEL SISTEMA

Bloques de recuperación en sistemas de tiempo real



SISTEMAS DE TIEMPO REAL

22

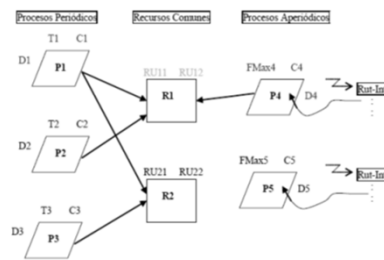
CARACTERÍSTICAS DEL SISTEMA

- MUCHOS CONTROLADORES PARA SISTEMAS INTEGRADOS UTILIZAN LA ESTRATEGIA DE SISTEMA EN UN CHIP (SOC: SYSTEM-ON-CHIP):
 - ◆ LA CPU, LA MEMORIA (INCLUYENDO LA CACHÉ), LA UNIDAD DE GESTIÓN DE MEMORIA (MMU: MEMORY-MANAGEMENT-UNIT) Y LOS PUERTOS PERIFÉRICOS ASOCIADOS (USB), ESTÁN CONTENIDOS EN UN ÚNICO CIRCUITO INTEGRADO.
 - ◆ ESTA ESTRATEGIA SOC SUELE SER MENOS COSTOSA QUE LA ORGANIZACIÓN ORIENTADA A BUS.
- LA CARACTERÍSTICA DEFINITORIA DE LOS SISTEMAS OPERATIVOS DE TIEMPO REAL (ESTRICTOS Y NO ESTRICTOS) CONSISTE EN DAR SOPORTE A LOS REQUISITOS DE TEMPORIZACIÓN DE LAS TAREAS EN TIEMPO REAL. PARA ELLO:
 - ◆ UTILIZAN ALGORITMOS DE PLANIFICACIÓN QUE PROPORCIONAN A LOS PROCESOS DE TIEMPO REAL LAS MÁS ALTAS PRIORIDADES.
 - ◆ MINIMIZAN EL TIEMPO DE RESPUESTA A LAS INTERRUPCIONES.

SISTEMAS DE TIEMPO REAL

20

CARACTERÍSTICAS DEL SISTEMA

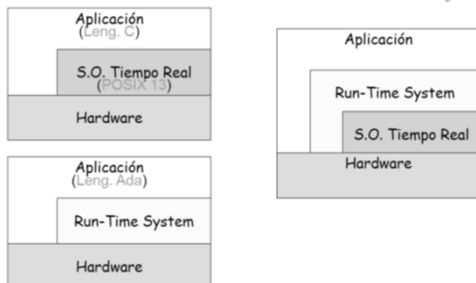


SISTEMAS DE TIEMPO REAL

23

CARACTERÍSTICAS DEL SISTEMA

Entornos de ejecución



SISTEMAS DE TIEMPO REAL

21

CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

SISTEMAS DE TIEMPO REAL

24

CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

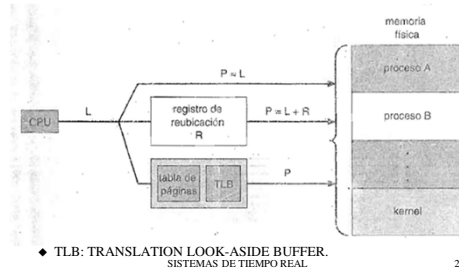
- LAS CARACTERÍSTICAS DE LOS NÚCLEOS DE TIEMPO REAL SON LIMITADAS PORQUE:
 - ◆ LA MAYORÍA DE LOS SISTEMAS EN TIEMPO REAL SIRVEN A UN ÚNICO PROPÓSITO:
 - NO INCLUYE LA NOCIÓN DE USUARIO: EL SISTEMA PERMITE POCAS TAREAS, QUE GENERALMENTE ESPERAN ENTRADAS QUE DEBEN RECIBIR DE DISPOSITIVOS HARDWARE (SENSORES, APARATOS DE VISIÓN, ETC.).
 - ◆ HAY RESTRICCIONES DE ESPACIO PARA PROCESADORES POTENTES Y GRANDES MEMORIAS:
 - ALGUNOS SISTEMAS ADMITEN SISTEMAS DE ARCHIVOS UTILIZANDO MEMORIA NO VOLÁTIL (NVRAM).
 - ◆ HAY LIMITACIONES DE COSTOS.

CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

- ◆ 2 - SE UTILIZA UNA TÉCNICA SIMILAR A LA DEL REGISTRO DE REUBICACIÓN DINÁMICA:
 - SE CONFIGURA UN REGISTRO DE REUBICACIÓN R CON LA UBICACIÓN DE MEMORIA EN LA QUE SE HA CARGADO UN PROGRAMA.
 - LA DIRECCIÓN FÍSICA P SE GENERA SUMANDO EL CONTENIDO DEL REGISTRO DE REUBICACIÓN R CON L .
 - LA VENTAJA ES QUE LA MMU (UNIDAD DE GESTIÓN DE MEMORIA) PUEDE TRADUCIR FÁCILMENTE LAS DIRECCIONES: $P = L + R$.
 - LA DESVENTA ES QUE NO EXISTE PROTECCIÓN DE MEMORIA ENTRE LOS PROCESOS.

CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

- LA MEMORIA VIRTUAL EN UN SISTEMA DE TIEMPO REAL
- TRADUCCIÓN DE DIRECCIONES EN LOS SISTEMAS DE TIEMPO REAL:



CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

- ◆ 3 - SE PROPORCIONA UNA FUNCIONALIDAD COMPLETA DE MEMORIA VIRTUAL:
 - LA TRADUCCIÓN DE DIRECCIONES USA TABLAS DE PÁGINAS Y UN BÚFER DE TRADUCCIÓN O TLD.
 - PERMITE CARGAR UN PROGRAMA EN CUALQUIER POSICIÓN DE MEMORIA.
 - PROPORCIONA PROTECCIÓN DE MEMORIA ENTRE LOS PROCESOS.
 - SE PODRÍAN IMPLEMENTAR LOS MECANISMOS DE PAGINACIÓN BAJO DEMANDA Y DE INTERCAMBIO UTILIZANDO MEMORIAS FLASH (NVRAM).
 - EJ.: LYNXOS Y ONCORE SYSTEMS.

CARACTERÍSTICAS DE UN KERNEL (NÚCLEO) DE TIEMPO REAL

- LA CPU GENERA LA DIRECCIÓN LÓGICA L QUE DEBE HACERSE CORRESPONDER CON LA DIRECCIÓN FÍSICA P .
- LAS PRINCIPALES TÉCNICAS SON LAS SIGUIENTES:
 - ◆ 1 - LA CPU GENERA DIRECTAMENTE LAS DIRECCIONES FÍSICAS:
 - ES EL MODO DE DIRECCIONAMIENTO REAL.
 - NO EMPLEA MECANISMOS DE MEMORIA VIRTUAL.
 - EQUIVALE A DECIR QUE P EQUIVALE A L .
 - UN PROBLEMA ES LA AUSENCIA DE PROTECCIÓN DE MEMORIA ENTRE LOS PROCESOS.
 - PUEDE REQUERIR QUE LOS PROGRAMADORES ESPECIFIQUEN LA UBICACIÓN FÍSICA DONDE DEBEN CARGARSE SUS PROGRAMAS EN MEMORIA.
 - LA VENTAJA ES QUE EL SISTEMA ES BASTANTE RÁPIDO.
 - ES COMÚN EN LOS SISTEMAS INTEGRADOS CON RESTRICCIONES DE TIEMPO REAL ESTRUCTAS.

IMPLEMENTACIÓN

IMPLEMENTACIÓN

- LAS PRINCIPALES CARACTERÍSTICAS NECESARIAS PARA IMPLEMENTAR UN S.O. DE TIEMPO REAL SON:
 - ◆ PLANIFICACIÓN APROPIATIVA BASADA EN PRIORIDADES.
 - ◆ KERNEL APROPIATIVO.
 - ◆ LATENCIA MINIMIZADA.
 - ◆ CONEXIÓN A RED DE DATOS (NO SIEMPRE).
- PLANIFICACIÓN BASADA EN PRIORIDADES
- EL PLANIFICADOR DEBE PERMITIR UN ALGORITMO BASADO EN PRIORIDADES CON APROPIACIÓN:
 - ◆ UN PROCESO QUE SE ESTÉ EJECUTANDO ACTUALMENTE EN LA CPU SERÁ DESALOJADO SI OTRO PROCESO DE MAYOR PRIORIDAD PASA A ESTAR DISPONIBLE PARA EJECUCIÓN.

IMPLEMENTACIÓN

- ◆ UTILIZAR MECANISMOS DE SINCRONIZACIÓN:
 - EL KERNEL SIEMPRE PUEDE SER APROPIATIVO:
 - CUALQUIER DATO DEL KERNEL QUE SE ESTÉ ACTUALIZANDO ESTARÁ PROTEGIDO FRENTE A POSIBLES MODIFICACIONES POR PARTE DEL PROCESO DE ALTA PRIORIDAD.
- MINIMIZACIÓN DE LA LATENCIA
- SE DENOMINA LATENCIA DEL SUCESO A LA CANTIDAD DE TIEMPO QUE TRANSCURRE:
 - ◆ DESDE EL MOMENTO QUE TIENE LUGAR EL SUCESO.
 - ◆ HASTA EL MOMENTO EN EL QUE SE LE DA SERVICIO.

IMPLEMENTACIÓN

- LOS SISTEMAS DE TIEMPO REAL ESTRUCTOS DEBEN ADEMÁS GARANTIZAR QUE LAS TAREAS DE TIEMPO REAL RECIBAN SERVICIO DE ACUERDO CON SUS REQUISITOS DE TEMPORIZACIÓN:
 - ◆ PUEDEN NECESITAR CARACTERÍSTICAS DE PLANIFICACIÓN ADICIONALES.
- KERNELS APROPIATIVOS
- UN KERNEL APROPIATIVO PERMITIRÁ DESALOJAR UNA TAREA QUE SE ESTÉ EJECUTANDO EN MODO KERNEL.
- EN LOS SISTEMAS DE TIEMPO REAL ESTRUCTOS RESULTA OBLIGATORIO UTILIZAR UN KERNEL APROPIATIVO:
 - ◆ ES PARA QUE UNA TAREA DE TIEMPO REAL NO DEBA ESPERAR UN PERÍODO DE TIEMPO ARBITRARIAMENTE LARGO, MIENTRAS HAYA OTRA TAREA ACTIVA EN EL KERNEL.

IMPLEMENTACIÓN

- LATENCIA DE UN SUCESO:
- DIFERENTES SUCESOS TIENEN DISTINTOS REQUERIMIENTOS DE LATENCIA:
 - ◆ SISTEMA ANTIBLOQUEO DE FRENOS: 3-5 MILLISEGUNDOS.
 - ◆ CONTROL DEL RADAR DE UN AVIÓN: 2-5 SEGUNDOS.

IMPLEMENTACIÓN

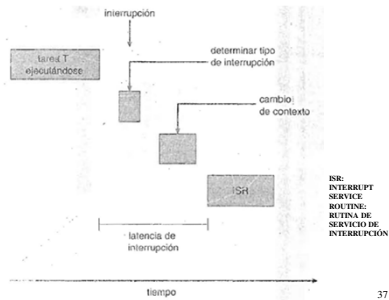
- LAS PRINCIPALES ESTRATEGIAS PARA LOGRAR UN KERNEL APROPIATIVO SON:
 - ◆ INSERTAR PUNTOS DE DESALOJO EN LAS LLAMADAS AL SISTEMA DE LARGA DURACIÓN:
 - EN CADA PUNTO DE DESALOJO SE COMPROBABA SI EXISTE LA NECESIDAD DE EJECUTAR UN PROCESO DE ALTA PRIORIDAD.
 - EN CASO DE QUE EXISTA SE PRODUCE UN CAMBIO DE CONTEXTO.
 - CUANDO EL PROCESO DE ALTA PRIORIDAD TERMINE DE EJECUTARSE, EL PROCESO INTERRUMPIDO CONTINUARÁ CON SU LLAMADA AL SISTEMA.
 - LOS PUNTOS DE DESALOJO SÓLO PUEDEN COLOCARSE EN PUNTOS SEGUROS DEL KERNEL:
 - SÓLO EN AQUELLOS PUNTOS DONDE LAS ESTRUCTURAS DEL KERNEL NO ESTÉN SIENDO MODIFICADAS.

IMPLEMENTACIÓN

- LOS TIPOS DE LATENCIA SON:
 - ◆ LATENCIA DE INTERRUPCIÓN.
 - ◆ LATENCIA DE DESPACHO.
- LATENCIA DE INTERRUPCIÓN
- ES EL PERÍODO DE TIEMPO QUE TRANSCURRE:
 - ◆ ENTRE LA LLEGADA DE UNA INTERRUPCIÓN A LA CPU.
 - ◆ EL INSTANTE EN QUE COMIENZA LA Rutina de Servicio de dicha interrupción.
- SE DEBE MINIMIZAR LA LATENCIA DE INTERRUPCIÓN.

IMPLEMENTACIÓN

■ LATENCIA DE INTERRUPCIÓN:



37

IMPLEMENTACIÓN

■ LA FASE DE CONFLICTO EN LA LATENCIA DE DESPACHO TIENE LOS SIGUIENTES COMPONENTES:

- ◆ EL **DESALOJO** DE CUALQUIER PROCESO QUE SE ESTÉ EJECUTANDO EN EL **KERNEL**.
- ◆ LA **LIBERACIÓN** POR PARTE DE LOS PROCESOS DE BAJA PRIORIDAD DE LOS **RECURSOS** NECESARIOS PARA EL PROCESO DE ALTA PRIORIDAD.
- SE PUEDE AFECTAR A LA LATENCIA DE DESPACHO CUANDO UN **PROCESO DE MAYOR PRIORIDAD** NECESITA LEER O MODIFICAR DATOS DEL **KERNEL** A LOS QUE ESTÉ ACTUALMENTE ACCEDIENDO UN **PROCESO DE MENOR PRIORIDAD**:
 - ◆ EL PROCESO DE **MAYOR PRIORIDAD** TENDRÁ QUE **ESPERAR** A QUE OTRO DE MENOR PRIORIDAD FINALICE CON EL RECURSO.
 - ◆ SE COMPLICA AÚN MÁS SI **EL PROCESO DE MENOR PRIORIDAD** ES **DESALOJADO** EN FAVOR DE OTRO PROCESO QUE TENGA UNA **PRIORIDAD MAYOR**.

SISTEMAS DE TIEMPO REAL

40

IMPLEMENTACIÓN

- **LATENCIA DE DESPACHO**
- ES LA **CANTIDAD DE TIEMPO** REQUERIDA PARA QUE EL **DESPACHADOR** DE PLANIFICACIÓN **DETENGA** UN PROCESO E **INICIE** OTRO.
- SE DEBE **MINIMIZAR** LA LATECIA DE DESPACHO:
 - ◆ SE DEBE TENER UN **KERNEL APROPIATIVO**.

SISTEMAS DE TIEMPO REAL

38

IMPLEMENTACIÓN

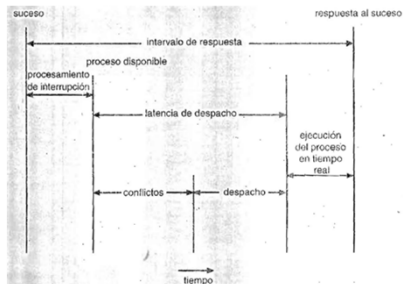
- ES EL PROBLEMA DE INVERSIÓN DE PRIORIDADES:
 - ◆ SE PUEDE RESOLVER CON EL **PROTOCOLO DE HERENCIA DE PRIORIDADES**:
 - TODOS LOS **PROCESOS** QUE ESTÉN **ACCEDIENDO** A **RECURSOS** QUE NECESITE **OTRO PROCESO DE MAYOR PRIORIDAD** **HEREDARÁN** ESA MAYOR PRIORIDAD HASTA QUE HAYAN **TERMINADO** CON LOS RECURSOS EN CUESTIÓN.
 - UNA VEZ QUE HAN TERMINADO SUS PRIORIDADES **REVIERTEN** A LOS VALORES ORIGINALES.

SISTEMAS DE TIEMPO REAL

41

IMPLEMENTACIÓN

■ LATENCIA DE DESPACHO:



SISTEMAS DE TIEMPO REAL

39

PLANIFICACIÓN DEL PROCESADOR

SISTEMAS DE TIEMPO REAL

42

PLANIFICACIÓN DEL PROCESADOR

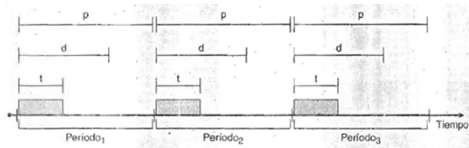
- SE CONSIDERARÁ LA PLANIFICACIÓN PARA LOS SISTEMAS DE TIEMPO REAL ESTRUCTOS.
- SE DEFINIRÁN CIERTAS CARACTERÍSTICAS DE LOS PROCESOS QUE HAY QUE PLANIFICAR.
- LOS PROCESOS SE CONSIDERAN PERIÓDICOS:
 - REQUIEREN LA CPU A INTERVALOS CONSTANTES (PERÍODOS).
 - CADA PROCESO TIENE UN TIEMPO DE PROCESAMIENTO FIJO T UNA VEZ QUE ADQUIERE LA CPU, UN PLAZO D ANTES DEL CUAL DEBE SER SERVIDO POR LA CPU Y UN PERÍODO P .
 - $0 \leq T \leq D \leq P$.

PLANIFICACIÓN DEL PROCESADOR

- PLANIFICACIÓN POR PRIORIDAD MONÓTONA EN TASA
- ESTE ALGORITMO (RATE-MONOTONIC SCHEDULING) PLANIFICA LAS TAREAS PERIÓDICAS UTILIZANDO UNA POLÍTICA DE PRIORIDAD ESTÁTICA CON APROPIACIÓN.
- SI SE ESTÁ EJECUTANDO UN PROCESO DE MENOR PRIORIDAD Y OTRO PROCESO DE MAYOR PRIORIDAD PASA A ESTAR DISPONIBLE PARA EJECUCIÓN:
 - ESTE PROCESO DESALOJARÁ AL DE MENOR PRIORIDAD.
- A CADA TAREA PERIÓDICA SE LE ASIGNA UNA PRIORIDAD QUE ESTÁ EN FUNCIÓN INVERSA A SU PERÍODO:
 - CUANTO MÁS CORTO SEA EL PERÍODO, MAYOR SERÁ LA PRIORIDAD.
 - CUANTO MÁS LARGO SEA EL PERÍODO, MENOR SERÁ LA PRIORIDAD.
- CONSISTE EN ASIGNAR UNA MAYOR PRIORIDAD A AQUELLAS TAREAS QUE REQUIEREN LA CPU MÁS A MENUDO.

PLANIFICACIÓN DEL PROCESADOR

- TASA PERIÓDICA:



- LA TASA DE UNA TAREA PERIÓDICA ES $1/P$.
- SE MUESTRA LA EJECUCIÓN DE UN PROCESO PERIÓDICO A LO LARGO DEL TIEMPO.
- LOS PLANIFICADORES PUEDEN APROVECHAR ESTA RELACIÓN Y ASIGNAR LAS PRIORIDADES DE ACUERDO CON EL PLAZO DE UN PROCESO PERIÓDICO A CON SU TASA REQUERIDA.

PLANIFICACIÓN DEL PROCESADOR

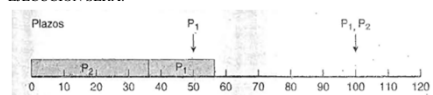
- PRESUPONE QUE EL TIEMPO DE PROCESAMIENTO DE UN PROCESO PERIÓDICO ES IGUAL EN CADA RÁFAGA DE LA CPU:
 - QUE CADA VEZ QUE UN PROCESO ADQUIERE LA CPU, LA DURACIÓN DE SU CORRESPONDIENTE RÁFAGA DE CPU ES LA MISMA.
- EJEMPLO: SE TIENE DOS PROCESOS P_1 Y P_2 CON PERÍODOS $p_1=50$ Y $p_2=100$ RESPECTIVAMENTE:
 - LOS TIEMPOS DE PRACESAMIENTO SON $t_1=20$ Y $t_2=35$.
 - EL PLAZO PARA CADA PROCESO REQUIERE QUE EL PROCESO COMPLETE SU RÁFAGA DE CPU ANTES DE QUE COMIENCE EL SIGUIENTE PERÍODO.
- PRIMERO HAY QUE VER SI ES POSIBLE PLANIFICAR ESTAS TAREAS DE MANERA QUE C/U SATISFAGA SUS REQUISITOS DE PLAZO.

PLANIFICACIÓN DEL PROCESADOR

- LO INUSUAL ES QUE UN PROCESO PUEDE TENER QUE ANUNCIAR AL PLANIFICADOR SUS REQUISITOS RELATIVOS AL PLAZO DE SERVICIO.
- UTILIZANDO LA TÉCNICA DEL ALGORITMO DE CONTROL DE ADMISIÓN EL PLANIFICADOR PUEDE:
 - ADMITIR EL PROCESO GARANTIZANDO QUE SE COMPLETARÁ EN EL TIEMPO PREFIJADO.
 - RECHAZAR LA SOLICITUD COMO IMPOSIBLE SI NO PUEDE GARANTIZAR QUE SE VAYA A DAR SERVICIO A LA TAREA ANTES DE ALCANZAR EL PLAZO.

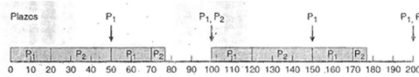
PLANIFICACIÓN DEL PROCESADOR

- SI SE MIDE LA UTILIZACIÓN DE CPU POR PARTE DE UN PROCESO P_i COMO LA RELACIÓN ENTRE SU RÁFAGA DE PROCESAMIENTO Y SU PERÍODO (t_i / p_i):
 - LA UTILIZACIÓN DE LA CPU POR PARTE DE P_1 SERÁ $20/50 = 0,40$; LA DE P_2 SERÁ $35/100 = 0,35$.
 - LA UTILIZACIÓN TOTAL DE LA CPU SERÁ DEL 75%.
 - PARCE QUE SÍ SE PUEDE PLANIFICAR ESTAS TAREAS DE TAL FORMA QUE AMBAS CUMPLAN CON SUS PLAZOS Y SIGAN DEJANDO A LA CPU UNA SERIE DE CICLOS DISPONIBLES.
- SI SE ASIGNA A P_2 UNA MAYOR PRIORIDAD QUE A P_1 LA EJECUCIÓN SERÁ:



PLANIFICACIÓN DEL PROCESADOR

- P_2 COMIENZA Y SE COMPLETA EN EL INSTANTE 35.
- EN 35 COMIENZA P_1 QUE TERMINA EN EL INSTANTE 55.
- EL PRIMER PLAZO PARA P_1 ERA EN EL INSTANTE 50.
- EL PLANIFICADOR HA HECHO QUE P_1 NO SE COMPLETE ANTES DEL PLAZO.
- UTILIZANDO UNA PLANIFICACIÓN POR PRIORIDAD MONÓTONA EN TASA Y ASIGNANDO A P_1 UNA MAYOR PRIORIDAD QUE A P_2 YA QUE EL PERÍODO DE P_1 ES MÁS CORTO QUE EL DE P_2 LA EJECUCIÓN SERÁ:

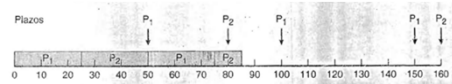


SISTEMAS DE TIEMPO REAL

49

PLANIFICACIÓN DEL PROCESADOR

- PROBLEMA DE NO CUMPLIMIENTO DE PLAZOS CON LA PLANIFICACIÓN POR PRIORIDAD MONÓTONA EN TASA:



SISTEMAS DE TIEMPO REAL

52

PLANIFICACIÓN DEL PROCESADOR

- P_1 COMIENZA PRIMERO Y TERMINA EN EL INSTANTE 20, CUMPLIENDO SU PRIMER PLAZO.
- P_2 COMIENZA A EJECUTARSE EN ESTE PUNTO Y SE EJECUTA HASTA EL INSTANTE 50:
 - ◆ ES DESALOJADO POR P_1 AUNQUE TODAVÍA FALTAN 5 MILISEGUNDOS PARA COMPLETAR SU RÁFAGA.
- P_1 COMPLETA SU RÁFAGA DE CPU EN EL INSTANTE 70:
 - ◆ EL PLANIFICADOR REANUDA LA EJECUCIÓN DE P_2 , QUE COMPLETA SU RÁFAGA DE CPU EN EL INSTANTE 75, CUMPLIENDO TAMBIÉN CON SU PRIMER PLAZO.
- EL SISTEMA QUEDARÁ INACTIVO HASTA EL INSTANTE 100, CUANDO SE VUELVA A PLANIFICAR LA EJECUCIÓN DE P_1 .
- LA PLANIFICACIÓN POR PRIORIDAD MONÓTONA EN TASA SE CONSIDERA ÓPTIMA, EN EL SENTIDO DE QUE SI UN CONJUNTO DE PROCESOS NO PUEDE PLANIFICARSE UTILIZANDO ESTE ALGORITMO, NO PODRÁ SER PLANIFICADO POR NINGÚN OTRO ALGORITMO QUE ASIGNE PRIORIDADES ESTÁTICAS.

SISTEMAS DE TIEMPO REAL

50

PLANIFICACIÓN DEL PROCESADOR

- PLANIFICACIÓN POR PRIORIDAD EN FINALIZACIÓN DE PLAZO
- EL ALGORITMO EDF (EARLIEST-DEADLINE-FIRST) ASIGNA DINÁMICAMENTE LAS PRIORIDADES DE ACUERDO CON LA FINALIZACIÓN DE LOS PLAZOS.
- CUANTO MÁS PRÓXIMO ESTÉ UN PLAZO, MAYOR SERÁ LA PRIORIDAD, Y CUANTO MÁS LEJANO ESTÉ EL PLAZO, MENOR SERÁ LA PRIORIDAD.
- UN PROCESO EJECUTABLE DEBE ANUNCIAR SUS REQUISITOS DE PLAZOS AL SISTEMA.
- PUEDE QUE SEA NECESARIO AJUSTAR LAS PRIORIDADES PARA REFLEJAR EL SIGUIENTE PLAZO DE ESE PROCESO QUE ACABA DE PASAR A SER EJECUTABLE.

SISTEMAS DE TIEMPO REAL

53

PLANIFICACIÓN DEL PROCESADOR

- EJEMPLO DE PROCESOS QUE NO PUEDEN PLANIFICARSE UTILIZANDO ESTE ALGORITMO.
- EL PROCESO P_1 TIENE PERÍODO $p_1=50$ Y UNA RÁFAGA DE CPU DE $t_1=25$.
- PARA P_2 LOS VALORES SON $p_2=80$ Y $t_2=35$.
- LA PLANIFICACIÓN ASIGNARÍA AL PROCESO P_1 UNA PRIORIDAD MÁS ALTA, YA QUE TIENE EL PERÍODO MÁS CORTO.
- LA UTILIZACIÓN TOTAL DE LA CPU SERÁ $(25/50) + (35/80) = 0,94$.
- PARECE LÓGICO QUE PUDIERAN PLANIFICARSE LOS DOS PROCESOS Y SEGUIR DEJANDO UN 6 % DE TIEMPO DE CPU DISPONIBLE.
- EN REALIDAD P_2 NO PODRÁ COMPLETAR SU RÁFAGA DE CPU ANTES DE SU PLAZO, EN EL INSTANTE 80.

SISTEMAS DE TIEMPO REAL

51

PLANIFICACIÓN DEL PROCESADOR

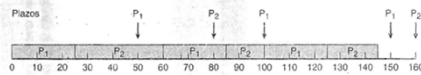
- PARA LOS PROCESOS DEL EJEMPLO ANTERIOR SE TENDRÁ:
- EL PROCESO P_1 LOGRA SU FINALIZACIÓN DE PLAZO; SU PRIORIDAD INICIAL ES MÁS ALTA QUE LA DEL PROCESO P_2 .
- EL PROCESO P_2 COMIENZA A EJECUTARSE.
- EL ALGORITMO PERMITE QUE P_2 CONTINUE EJECUTÁNDOSE; P_2 TIENE AHORA MAYOR PRIORIDAD QUE P_1 PORQUE SU SIGUIENTE PLAZO (EN EL INSTANTE 80) ESTÁ MÁS PRÓXIMO QUE EL DE P_1 (EN EL INSTANTE 100).
- LOS DOS PROCESOS PUEDEN CUMPLIR CON SU PRIMER PLAZO.
- P_1 COMIENZA A EJECUTARSE DE NUEVO EN EL INSTANTE 60 Y COMPLETA SU SEGUNDA RÁFAGA DE CPU EN EL INSTANTE 85, CUMPLIENDO TAMBIÉN CON SU SEGUNDO PLAZO EN EL INSTANTE 100.
- P_2 COMENZARÁ A EJECUTARSE EN ESTE PUNTO PARA SER DESALOJADO POR P_1 AL COMIENZO DE SU SIGUIENTE PERÍODO EN EL INSTANTE 100.

SISTEMAS DE TIEMPO REAL

54

PLANIFICACIÓN DEL PROCESADOR

- P_2 ES DESALOJADO PORQUE P_1 TIENE UNA FINALIZACIÓN DE PLAZO ANTERIOR (INSTANTE 150) QUE P_2 (INSTANTE 160).
- EN EL INSTANTE 125, P_1 COMPLETA SU RÁFAGA DE CPU Y P_2 REANUDA SU EJECUCIÓN, FINALIZANDO EN EL INSTANTE 145 Y CUMPLIENDO TAMBIÉN CON SU PLAZO.
- EL SISTEMA ESTARÁ INACTIVO HASTA EL INSTANTE 150, EN EL QUE SE PLANIFICA DE NUEVO LA EJECUCIÓN DE P_1 .



PLANIFICACIÓN DEL PROCESADOR

- EL ALGORITMO DEBE TRABAJAR EN CONJUNCIÓN CON UNA **POLÍTICA DE CONTROL DE ADMISIÓN** PARA GARANTIZAR QUE CADA APLICACIÓN RECIBA SU CUOTA DE TIEMPO ASIGNADA.
- LA **POLÍTICA DE CONTROL DE ADMISIÓN SÓLO ADMITIRÁ** A UN CLIENTE QUE SOLICITE UN NÚMERO CONCRETO DE CUOTAS **SI HAY SUFICIENTES CUOTAS DISPONIBLES**.
- EN EL EJEMPLO SE HAN ASIGNADO $50+15+20=85$ CUOTAS DEL TOTAL DE 100:
 - ◆ SI UN NUEVO PROCESO **D** SOLICITARA 30 CUOTAS EL CONTROLADOR DE ADMISIÓN **DENEGARÍA** A **D** LA ENTRADA EN EL SISTEMA.

PLANIFICACIÓN DEL PROCESADOR

- LA PLANIFICACIÓN EDF NO REQUIERE:
 - ◆ QUE LOS PROCESOS SEAN PERIÓDICOS.
 - ◆ QUE NECESITEN UNA CANTIDAD CONSTANTE DE TIEMPO DE CPU POR CADA RÁFAGA DE EJECUCIÓN.
- EL ÚNICO REQUISITO ES QUE CADA PROCESO INDIQUE AL PLANIFICADOR SU **PRÓXIMO PLAZO** EN EL MOMENTO DE PASAR A SER EJECUTABLE.
- LA PLANIFICACIÓN EDF RESULTA ÓPTIMA DESDE EL PUNTO DE VISTA **TEÓRICO**:
 - ◆ PERMITE PLANIFICAR LOS PROCESOS DE MODO QUE C/U PUEDA CUMPLIR CON SUS REQUISITOS RELATIVOS A LOS PLAZOS DE EJECUCIÓN Y LA **UTILIZACIÓN** DE LA CPU SERÁ DEL 100%.
- EN LA **PRÁCTICA** ES IMPOSIBLE CONSEGUIR ESTE NIVEL DE UTILIZACIÓN DE LA CPU DEBIDO AL COSTE DE:
 - ◆ LOS CAMBIOS DE CONTEXTO ENTRE PROCESOS.
 - ◆ EL COSTE DEL **TRATAMIENTO DE LAS INTERRUPCIONES**.

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

PLANIFICACIÓN DEL PROCESADOR

- **PLANIFICACIÓN CON CUOTA PROPORCIONAL**
- ESTE ALGORITMO FUNCIONA ASIGNANDO T CUOTAS ENTRE TADAS LAS APLICACIONES.
- CADA APLICACIÓN PUEDE RECIBIR N CUOTAS DE TIEMPO, GARANTIZANDO ASÍ QUE LA APLICACIÓN TENGA UN N/T DEL TIEMPO TOTAL DEL PROCESADOR.
- **EJEMPLO:** SE SUPONE QUE HAY $T=100$ CUOTAS QUE HAY QUE DIVIDIR ENTRE TRES PROCESOS, A, B Y C.
- SE ASIGNAN 50 CUOTAS A A, 15 A B Y 20 A C.
- ESTE ESQUEMA **ASEGURA** QUE A TENDRÁ UN 50% DEL TIEMPO TOTAL DE PROCESADOR, B TENDRÁ UN 15% Y C TENDRÁ UN 20%.

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- **VRTX: Versatile Real Time Executive**
- Disponible para Intel, Motorola, VME
- Multitarea: Dispone de mecanismos para satisfacer los requisitos de TR de la mayoría de las aplicaciones empotradas
- Mecanismos de exclusión mutua, señalización y comunicación

VME: VERSA MODULE EUROPE - MOTOROLA

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- VX Works
- Es de Wind River. Más económico que VRTX
- Utilitarios para depuración simbólica, comunicación a través de redes por medio de TCP/IP y sockets
- Estructura jerárquica

SISTEMAS DE TIEMPO REAL

61

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

RTEMS (Real time for military systems)

Altas prestaciones:

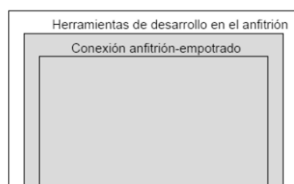
- multitarea
- entorno homogéneo para mono y multiprocesamiento
- planificación con prioridades activadas por eventos

SISTEMAS DE TIEMPO REAL

64

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- Micronúcleo proporciona los servicios de:
 - multitarea
 - comunicación entre procesos
 - sincronización

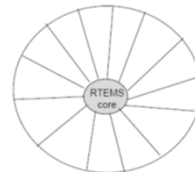


SISTEMAS DE TIEMPO REAL

62

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- Planificación monotónica en frecuencia opcional
- comunicación entre tareas y sincronización
- herencia de prioridades
- gestión de interrupciones
- gestión dinámica de memoria
- alto nivel de reconfigurabilidad



SISTEMAS DE TIEMPO REAL

65

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- RT-Mach
- Versión en tiempo real de MACH
- Incluye:
 - Gestión de procesos ligeros basada en el modelo ARTS, con características de TR
 - Planificador activado por eventos integrado en el núcleo
 - Objetos residentes en memoria
 - Herramientas de tiempo real: planificador 1-2-3 y monitor avanzado de tiempo real (ARM).

SISTEMAS DE TIEMPO REAL

63

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- SPRING
- Sistema operativo de tiempo real desarrollado en la Univ. de Manchester
- Trata de resolver algunos conceptos equivocados y deficiencias tradicionales en SO de TR. Es un sistema activado por eventos de gran flexibilidad y muy predecible.
- Se clasifican las tareas sobre la base del coste relativo del incumplimiento del tiempo de respuesta:
 - » Tareas críticas
 - » Tareas esenciales
 - » Tareas no esenciales

SISTEMAS DE TIEMPO REAL

66

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- QNX
- Proporciona un entorno completo de tiempo real para redes de mono o multiprocesadores
- Arquitectura: Micronúcleo rodeado por un conjunto de procesos opcionales que brindan compatibilidad con POSIX y UNIX
- Servicios del micronúcleo: Comunicación entre procesos, planificación y tratamiento de interrupciones de bajo nivel.
- Fuera del micronúcleo la funcionalidad se proporciona mediante servidores.
 - Obligatorio: Servidor de procesos
 - Opcionales: Gestor de sistemas de archivos
 - Gestor de dispositivos
 - Gestor de redes

SISTEMAS DE TIEMPO REAL

67

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- MARS
- Maintainable Real Time System
- Sistema distribuido de tiempo real tolerante a fallas, para control de procesos.
- Desarrollado en la Universidad Técnica de Viena
- **Claves:**
 - Tolerancia a fallas
 - Planificación estática
 - Reparación dinámica
 - Manejo de redes redundantes

SISTEMAS DE TIEMPO REAL

70

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

HARTOS

Desarrollado para la arquitectura HARTS (red hexagonal de multiprocesadores de memoria compartida)

El núcleo proporciona una interfaz uniforme para comunicaciones entre procesos de tiempo real

SISTEMAS DE TIEMPO REAL

68

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- SOTR Orientados a Objetos
- CHAOS: Concurrent Hierarchical Object System
- Sistema basado en objetos
- El núcleo ofrece los sistemas básicos y sobre el mismo se desarrollan sistemas de tiempo real

SISTEMAS DE TIEMPO REAL

71

EJEMPLOS DE SOTR (SISTEMAS OPERATIVOS DE TIEMPO REAL)

- CHORUS
- Núcleo para sistemas distribuidos. Similar a MACH.
- Reescrito para que sirva como núcleo en sistemas de tiempo real.
- **Mejoras:**
 - Núcleo mínimo (10-100Kb) con servicios básicos que permiten implementar servidores UNIX, POSIX, etc.
 - Procesos de tiempo real críticos.
 - Personalizaciones, construidas sobre el micronúcleo que permiten integrar CHORUS con otros sistemas (ONX, VRTX, UNIX, OS2, etc.)
 - Distribución transparente conectividad total entre los núcleos del sistema.
 - Interfaces abiertas y servicios estándares.

SISTEMAS DE TIEMPO REAL

69